

ASSIGNMENT – 13.2

2303A51195

BATCH : 04

LAB – 13 : Code Refactoring – Improving Legacy Code with AI Suggestions.

Task – 01 : Refactoring – Eliminating Repetitive Logic.

Prompt : Use AI to analyze the Python code and identify repeated calculations for area and perimeter. Refactor the code by creating a reusable function with proper docstrings while ensuring the output remains the same.

Original Code & Output :



```
Commands + Code + Text Run all
TASK 01
ORIGINAL CODE
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))

length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))

Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
REFACTORED CODE
Variables Terminal 1:30 PM Python 3
```

Refactored Code & Output :



```
Commands + Code + Text Run all
REFACTORED CODE
def calculate_rectangle(length, width):
    area = length * width
    perimeter = 2 * (length + width)
    print("Area:", area)
    print("Perimeter:", perimeter)

calculate_rectangle(8, 4)
calculate_rectangle(12, 6)

Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
```

Explanation :

The repeated calculations for area and perimeter were moved into a reusable function called `calculate_rectangle`. This improves code readability and reduces duplication. The program still produces the same output but is easier to maintain.

Task – 02 : Refactoring – Modularizing Inline Calculations.

Prompt : Use AI to refactor the Python code by identifying repeated discount calculations and converting them into a reusable function with proper documentation.

Original Code & Output :



```
TASK 02

ORIGINAL CODE

amount = 1000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)

amount = 2000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)

Final Amount: 900.0
Final Amount: 1800.0
```

Refactored Code & Output :



```
REFACTORED CODE

def calculate_final_amount(amount):
    discount = amount * 0.10
    final_amount = amount - discount
    print("Final Amount:", final_amount)

calculate_final_amount(1000)
calculate_final_amount(2000)

Final Amount: 900.0
Final Amount: 1800.0
```

Explanation :

The repeated discount calculation logic was moved into a function called `calculate_final_amount`. This makes the code modular and reusable for different input values. It also improves readability and reduces redundant code.

Task – 03 : Refactoring – Improving Loop and Conditional Performance.

Prompt : Use AI to optimize the Python program by replacing inefficient nested loops with a faster lookup structure while keeping the output unchanged.

Original Code & Output :



```
TASK 03  
ORIGINAL CODE  
[10] students = ["Anil", "Bhavya", "Charan", "Divya"]  
check_list = ["Charan", "Esha", "Bhavya"]  
for name in check_list:  
    exists = False  
    for student in students:  
        if name == student:  
            exists = True  
    if exists:  
        print(name, "found")  
    else:  
        print(name, "not found")  
--- Bhavya found
```

Refactored Code & Output :



```
REFACTORED CODE  
[10] students = ["Anil", "Bhavya", "Charan", "Divya"]  
check_list = ["Charan", "Esha", "Bhavya"]  
  
student_set = set(students)  
  
for name in check_list:  
    if name in student_set:  
        print(name, "found")  
    else:  
        print(name, "not found")  
--- Charan found  
--- Esha not found  
--- Bhavya found
```

Explanation :

The nested loops were replaced with a set data structure, which provides faster lookup operations. Instead of checking each student repeatedly, membership is checked directly using `in`. This improves performance from $O(n^2)$ to $O(n)$.

Task – 04 : Refactoring – Replacing Hardcoded Values with Constants.

Prompt : Use AI to identify hardcoded numerical values in the simple interest calculation and replace them with descriptive constants to improve maintainability.

Original/Refactored Code & Output :

```
Q Commands + Code + Text ▶ Run all ✓ RAM 100% ✓ 1:48 PM Python 3
```

TASK 04

ORIGINAL CODE

```
171 print("Simple Interest:", (1000 * 2 * 5) / 100)
172 print("Simple Interest:", (1000 * 2 * 5) / 100)
```

Simple Interest: 100.0
Simple Interest: 200.0

REFACTORED CODE

```
171 RATE = 2
172 TIME = 5
173 PERCENT = 100
174 def calculate_simple_interest(principal):
175     interest = (principal * RATE * TIME) / PERCENT
176     print("Simple Interest:", interest)
177 calculate_simple_interest(1000)
178 calculate_simple_interest(2000)
```

Simple Interest: 100.0
Simple Interest: 200.0

Variables Terminal + 1:48 PM Python 3

Explanation :

Hardcoded values such as rate, time, and percentage were replaced with named constants (RATE, TIME, PERCENT). This makes the code easier to understand and modify. If the rate or time changes, it only needs to be updated once.

Task – 05 : Refactoring – Enhancing Variable Naming Code Clarity.

Prompt : Use AI to improve code readability by replacing unclear variable names with meaningful ones and adding comments explaining the logic.

Original/Refactored Code & Output :

```
Q Commands + Code + Text ▶ Run all ✓ RAM 100% ✓ 1:52 PM Python 3
```

ORIGINAL CODE

```
171 x = 50
172 y = 30
173 z = x + y
174 print(z)
```

80

REFACTORED CODE

```
171 # Define two numbers
172 num1 = 50
173 num2 = 30
174
175 # Calculate the sum of the numbers
176 total_sum = num1 + num2
177
178 # Display the result
179 print(total_sum)
```

80

Variables Terminal + 1:52 PM Python 3

Explanation :

The variables x, y, and z were renamed to num1, num2, and total_sum to improve clarity. Comments were added to explain each step of the code. This makes the program easier to understand for other developers.

THANK YOU!!